DSA4213 Assignment 1: Does Better Next-Token Prediction Give Better Representations?

Zhao Richard (code: `README.md` in the submitted repository)

1 Question and setup

Comparison question. *Does a language model (LM) that predicts the next token better also provide better features for a downstream task?* Part I ranks four LM families by perplexity (PPL); Part II changes only the LSTM's input embeddings; Part III freezes every LM and asks whether its PPL rank predicts its rank on AG News topic classification.

Corpus and preprocessing. WikiText-2 (`wikitext-2-v1`, official splits; 2.08M / 216k / 244k tokens). The v1 release is already tokenised with rare words mapped to `<unk>`; I lowercase (to match uncased GloVe), drop empty lines and append `<eos>` to every line. The vocabulary is all training types plus `<pad>`, `<unk>`, `<eos>` (28,913 types, built from train only and shared by every model). Lowercasing creates no new validation/test OOVs; `<unk>` is 6.2% of test tokens and is scored like any other token.

Common evaluation. Each split is cut into non-overlapping 128-token windows; every test token except the first is a target exactly once (244,101 targets) and is predicted only from the preceding tokens *inside its window* (state reset per window, also for the n-gram). The last window is right-padded and padded targets are excluded from the loss. Model selection uses validation loss only; the test set is used once.

Models. (i) *KN n-gram*: my own interpolated Kneser–Ney (one discount per order, $D_k = n_1/(n_1 + 2n_2)$, continuation counts for lower orders, uniform base); $n = 4$ chosen on validation (val PPL 274.8/235.8/229.3 for $n = 2/3/4$). (ii) *RNN* and (iii) *LSTM*: 300-d embeddings, 2 layers of 512 units, linear softmax. (iv) *Transformer*: decoder-only, pre-LN, 4 layers, 6 heads, $d = 300$, FFN 1200, learned positions, upper-triangular causal mask (a unit test confirms that editing tokens $\geq t$ leaves all logits $< t$ unchanged for all three neural models). All neural models use 300-d randomly initialised trainable embeddings (PyTorch default $\mathcal{N}(0, 1)$), dropout 0.3, and an *identical* procedure: context 128, batch 32, 20 epochs (≈10.1k steps, 41.5M target tokens), AdamW (lr 10^{-3}, wd 0.01), 300 warm-up steps then cosine decay, gradient clipping 1.0, best-validation checkpoint. Hardware: one RTX 5070 Ti Laptop GPU (12 GB) under WSL2, PyTorch 2.11; seed 42.

2 Part I: four language-model families

Model	Test PPL	Val PPL	Params (M)	Non-emb (M)	Train s (s/epoch)	Train k tok/s	Infer k tok/s	Gen ms/tok	Rep-3 greedy	Dist-2 top-k
KN 4-gram	216.5	229.3	5.8†	–	7 (CPU)	–	450 (CPU)	1.4	0.55	0.93
RNN	153.4	165.6	24.4	0.94	423 (21/ep)	105	274	1.9	0.34	0.92
LSTM	135.0	143.9	27.3	3.77	487 (23/ep)	101	273	1.8	0.43	0.92
Transformer	131.0	140.4	21.8	4.37	584 (29/ep)	126	484	2.3	0.41	0.90

Table 1: Part I. *Train s*: wall-clock of the training passes (median s/epoch). *Train/Infer k tok/s*: controlled benchmark (all models timed round-robin, 5 rounds, median; batch 32×128). *Gen*: greedy generation latency at batch 1 without caching. *Rep-3*: share of repeated trigrams in 40-token greedy continuations; *Dist-2*: distinct bigrams in top-k samples (5 prompts ×10). †stored n-gram count entries, not trainable parameters.

Prediction quality. Observation. All three neural LMs beat the KN 4-gram by a large margin (test PPL 216.5 → 131–153). Transformer (131.0) ≈ LSTM (135.0) < RNN (153.4). The Transformer–LSTM gap (3%) is of the same order as the seed-to-seed variation I measured for the LSTM (std (pending) PPL, Part II), so I treat these two as roughly tied; the RNN gap is far larger. **Possible explanation.** The n-gram can only use 3 tokens of context and backs off on the many unseen 4-grams, whereas the neural models share statistical strength through embeddings and use up to 127 tokens. The RNN has only 0.94M non-embedding parameters (vs. 3.8M/4.4M) and a recurrence without gating, so it plausibly cannot retain information over long spans.

Learning behaviour. Observation. (Fig. 1) The RNN improves fastest in epoch 1 but plateaus earliest (val CE 5.11). Validation loss flattens after ≈12 epochs for the LSTM (best epoch 19, then flat) while the Transformer is still improving slightly at epoch 20. The gap between validation and training loss grows steadily for all models (0.45–0.56 nats at epoch 20) without validation loss turning upward, i.e. the models memorise

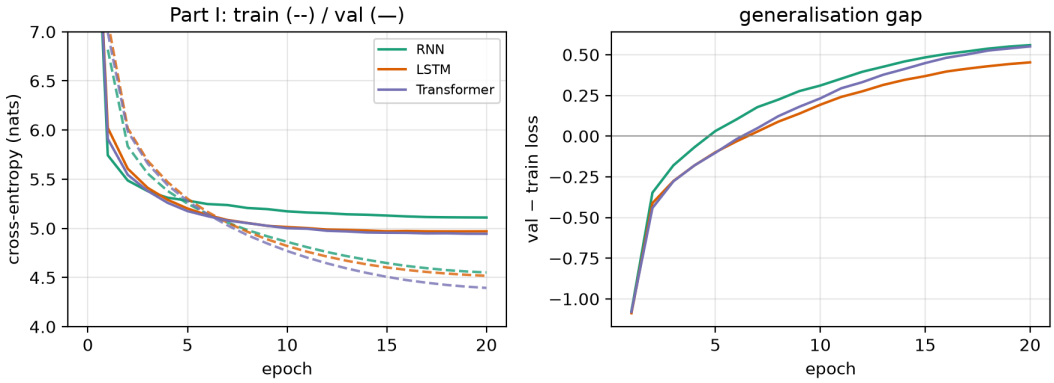


Figure 1: Part I training (dashed, measured with dropout) and validation (solid) cross-entropy, and the validation–training gap.

the 2M-token corpus but the cosine schedule and dropout keep this from hurting validation within 20 epochs. **Possible explanation.** The Transformer's continued improvement suggests it is the most budget-limited model; its lower training loss at equal validation loss means it fits the training set more easily.

Cost. Observation. In the controlled benchmark the Transformer is the fastest to train (126k tok/s vs. 101–105k) and to score text (484k vs. 273k tok/s), because all positions are processed in parallel, whereas RNN/LSTM steps are sequential. In batch-1 generation without caching the order flips (2.3 vs. 1.8 ms/token), since the Transformer re-encodes the entire prefix every step. The KN model trains in 7 s on one CPU thread. Sequential training wall-clock gives a misleading picture: the Transformer appears slowest (29 s/epoch) only because it was trained third, when the laptop GPU had throttled (epoch times of the *same* model drift from 21 to 33 s across the session). I therefore report the round-robin benchmark as the cost comparison.

Generated text. Observation. (Table 2; all models use the same prompts, 40 tokens, `<unk>` banned.) Under greedy decoding every model falls into loops; the KN model loops most (55% repeated trigrams, e.g. *"the first to be released in the united states , and"* repeated), the RNN least (34%). With top-k sampling, repetition almost vanishes for all models. The n-gram produces locally plausible 3–4-word fragments that do not form clauses (*"but i just want to be with them , and the first"*); the LSTM and Transformer produce grammatical clauses that stay on the war/military topic of the prompt for a full sentence (*"the royal fleet was an apprentice officer on the main naval brigade"*), but consistency breaks down after one or two sentences and facts are invented. All neural models learned WikiText layout (`= = heading = =`, `<eos>`). **Possible explanation.** Lower PPL clearly buys local fluency, but none of the models has enough data or capacity for discourse-level coherence; greedy loops are a decoding artefact shared by all model families.

Prompt: *during the second world war ,* (top-k=40, T=0.8 unless noted; first sample, first 24 tokens)	
KN 4-gram	and , unlike the most controversial of all the millennium stadium , but i just want to be with them , and the first . . .
RNN	and did not have two different types of the solar family . ⟨eos⟩in the late 20th century , many archaeologists were not only . . .
LSTM	the royal fleet was an apprentice officer on the main naval brigade . he was appointed adjutant 's flagship by the british navy , . . .
Transformer	the company took place off their first position in australia , where they suffered a new zealand brigade , they had been unable to . . .
LSTM+GloVe	the largest had been estimated and was considered a high @-@ time @-@ known @-@ scale @-@ time @-@ long area . the first . . .
KN 4-gram (greedy)	and the first to be released in the united states , and the first to be released in the united states , and the . . .
Transformer (greedy)	the first world war ii was the first world war ii . the war was the first world war ii in the war , . . .

Table 2: Matched continuations (more in `results/generations.md`).

3 Part II: input-embedding ablation (LSTM)

Design. I ablate the LSTM because it is the best recurrent model and its embedding is a clean, separable input. The Part I LSTM is setting 1 (identical configuration). Setting 2: my own Skip-gram vectors (gensim, 300-d, window 5, 5 negatives, 10 epochs) trained on the WikiText-2 *training split only* (cost: 82 s on one CPU thread, not counted in LM training time); they cover 100% of the vocabulary. Setting 3: GloVe 6B 300d (Wikipedia 2014 + Gigaword 5, 6B tokens). After mapping WikiText markers to GloVe tokens (`@-@`→`-`, `@,@`→`,`, `'t`→`n't`),

GloVe covers 96.2% of types and 95.9% of training tokens; the 1,098 missing types (mostly `<unk>`, `<eos>`, rare names) get $\mathcal{N}(0, \sigma^2)$ vectors with σ the std of the GloVe table (seed 42); `<pad>` is zero. In settings 2–3 the *entire* table, including these random rows, is frozen; everything else is retrained from scratch with the Part I procedure. Runs were repeated with seeds 1 and 2.

Input embedding	Cover.	Train. (M)	Test PPL seed 42	Test PPL 3 seeds	Val PPL ep 1	Val PPL ep 3	Ep. to best+.05	Gap ep 20	Step-loss std
Random $\mathcal{N}(0,1)$, trainable	–	27.3	135.0	–	412	224	10	0.45	0.127
Own Skip-gram, frozen	100.0%	18.6	130.4	–	559	239	11	0.40	0.123
GloVe 6B, frozen	96.2%	18.6	133.9	–	564	237	11	0.33	0.122

Table 3: Part II. *Cover.*: vocabulary types with a pretrained vector. *3 seeds*: mean ± std over seeds 42, 1, 2. *Ep. to best+.05*: first epoch whose val loss is within 0.05 nats of the run's best. *Gap*: val − train loss at epoch 20. *Step-loss std*: mean within-epoch std of mini-batch loss.

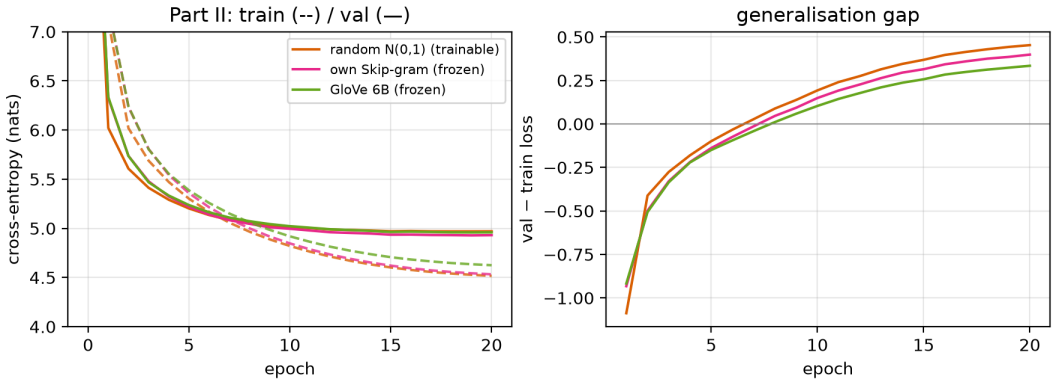


Figure 2: Part II learning curves (all LSTM, identical except the input-embedding table).

Results. Observation. (Table 3, Fig. 2) **PART2-RESULTS-PLACEHOLDER**

Why does "trainable" not win? Embedding drift. Observation. Measuring the cosine between each word's initial and final vector shows that the default-initialised trainable embeddings barely move: mean cosine 0.98 (0.94 for the 1,000 most frequent words) for the LSTM, and the same for the RNN (0.98) and Transformer (0.98). Their nearest neighbours are essentially random (*france* → outreach, blazers, …; Appendix Table 5), whereas Skip-gram and GloVe give clear semantic neighbours. **Possible explanation.** With $\mathcal{N}(0,1)$ initialisation, vectors have norm ≈17, so AdamW updates of size $\approx 10^{-3}$ over 10k steps (and only when a word occurs) leave them near their random start; the "trainable" setting therefore behaves like a *frozen random* table, and the LSTM's input weights do the adaptation. **SUPPL-PLACEHOLDER**

Interpretation. Two factors are confounded in the public-embedding setting: trainability (both pretrained settings are frozen) and extra data (GloVe saw ∼3,000× more text, including newswire). Comparing settings 2 and 3 holds trainability fixed and isolates the data/domain effect: for *predicting WikiText-2*, in-domain vectors trained on the same 2M tokens are at least as good as GloVe, so the extra data does not help this LM objective. The extra data does matter downstream (Part III).

4 Part III: frozen LM representations for AG News

Task and data. AG News 4-way topic classification (World/Sports/Business/Sci-Tech): a stratified 20k subset of the official training split (5k per class) and the full official test split (7,600). AG News is tokenised with a rule-based tokenizer reproducing WikiText conventions (e.g. `didn 't`, `3 @.@ 5`). OOV w.r.t. the WikiText-2 vocabulary is 7.1% of test tokens, about the same as SST-2 (7.0%), so coverage does not favour either dataset; I chose AG News because its encyclopaedic/news register is close to Wikipedia and its large labelled test set gives tight comparisons (SST-2's test labels are not public).

Representations. Inputs are truncated to 128 tokens, right-padded, and passed through each frozen LM. *Last*: top-layer state at the last real token. *Mean*: average of top-layer states over real tokens (padding masked). *Emb*: average of the LM's *input* embeddings over real tokens (no context; a reference showing what the contextual layers add). Classifier: standardisation + multinomial logistic regression, C chosen from $\{10^{-3}, \ldots, 1\}$ by macro-F1 on a 10% validation split of the training subset, then refit on all 20k. Identical for every representation; the LMs are never fine-tuned.

Frozen LM	LM test PPL	Last state acc / macro-F1	Mean of states acc / macro-F1	Mean of input emb. acc / macro-F1
RNN	153.4	0.617 / 0.616	0.705 / 0.705	0.755 / 0.754
LSTM	135.0	0.638 / 0.635	0.701 / 0.700	0.733 / 0.733
Transformer	131.0	0.719 / 0.718	0.767 / 0.766	0.770 / 0.769
LSTM + Skip-gram (frozen)	130.4	0.776 / 0.776	0.803 / 0.803	0.824 / 0.823
LSTM + GloVe (frozen)	133.9	0.790 / 0.790	0.846 / 0.846	0.878 / 0.878

Table 4: Part III test accuracy / macro-F1 on 7,600 AG News test articles. With n=7,600 the standard error of an accuracy near 0.8 is ≈0.005; exact McNemar tests are in `results/downstream/error_stats.json`.

Which representation works better? Observation. (Table 4) (1) Pooling: *mean* > *last* for every LM (e.g. LSTM 0.701 vs. 0.638, $p < 10^{-28}$). (2) Among the Part I models (same random embeddings), the Transformer is clearly best (0.767 vs. 0.701 for the LSTM, McNemar $p < 10^{-37}$), while the LSTM and RNN are indistinguishable (p=0.44) despite a 18-point PPL difference. (3) The frozen pretrained-embedding LSTMs give by far the best features (GloVe 0.846, Skip-gram 0.803). (4) Surprisingly, the context-free *emb* average matches or beats the contextual *mean* for every model (Transformer 0.770 vs. 0.767, p=0.44; GloVe-LSTM 0.878 vs. 0.846). **Possible explanation.** Topic is largely a bag-of-words property; next-token training makes the top layer encode what is needed to predict the *next* word (syntax, local continuation), which discards some lexical identity, and the last state is dominated by the final tokens (often punctuation). The pretrained vectors already place topical words close together; GloVe additionally saw Gigaword newswire, AG News's own domain.

Does PPL rank agree with downstream rank? Observation. Only weakly. Across the five LMs the Spearman correlation between test PPL and accuracy is ρ=-0.6 for mean pooling (p=0.28, n=5) and -0.7 for last-state pooling. The best LM by PPL is the Skip-gram LSTM, but the best downstream is the GloVe LSTM with a *worse* PPL; the LSTM beats the RNN by 18 PPL points but not downstream. The downstream ranking follows the quality of the *input embeddings* (GloVe > Skip-gram > barely-trained random) far more than PPL.

Error analysis. The dominant confusion for all models is Business↔Sci/Tech (515 of the GloVe-LSTM's 1,170 errors), e.g. *"Dell exits low-end China consumer PC market"* (gold Sci/Tech) or *"IBM posts broad Q3 revenue growth"* (gold Sci/Tech), which are about technology companies' finances—arguably ambiguous labels. 532 test articles are misclassified by all five LMs; many are of this kind or have debatable gold labels (*"Stocks climb on strong economic data"* labelled World). Where GloVe succeeds and the random-embedding LSTM fails, the decisive words are usually proper nouns rare in WikiText-2 but frequent in news (*Isinbayeva, DaimlerChrysler, Linux, Dalai Lama*). Accuracy rises with article length (GloVe-LSTM 0.82 on the shortest quartile vs. 0.85–0.86 otherwise), but is *not* lower for articles with more `<unk>` tokens (Appendix), suggesting that missing words are compensated by the remaining context. Per-class recall is lowest for Business for every model.

5 Connecting the parts, limitations, and lessons

Answer to the question. Within one embedding setting, a better LM (Transformer vs. RNN/LSTM) did produce better features, but PPL is a poor predictor overall: the three LSTMs of Part II differ by ≤5 PPL yet span 14 accuracy points, because what matters for topic classification is lexical semantics stored in the input embeddings, which next-token training on 2M tokens barely shaped (Part II drift analysis).

Limitations. (1) One corpus of 2M tokens and a fixed 20-epoch budget; the Transformer was still improving, so its PPL is likely under-estimated. (2) The default $\mathcal{N}(0, 1)$ embedding initialisation made "trainable" embeddings nearly static; this affects every Part I model equally but changes the meaning of the trainable-vs-frozen comparison (addressed by the supplementary run). (3) Only three seeds and only for the Part II LSTMs; Transformer vs. LSTM PPL is within seed noise. (4) Timing on a laptop GPU is affected by thermal throttling; the round-robin benchmark mitigates but does not remove this. (5) Part III uses a linear probe on frozen features; fine-tuning could change the ranking. (6) GloVe's newswire data overlaps AG News's domain, so its downstream advantage mixes data quantity and domain match.

Lessons. Report cost with a controlled benchmark, not sequential wall-clock; check what "trainable" parameters actually learn; and evaluate representations directly rather than assuming PPL transfers.

Tools and sources. WikiText-2 (Merity et al., 2017), GloVe (Pennington et al., 2014), AG News (Zhang et al., 2015), SST-2 (Socher et al., 2013) via HuggingFace `datasets` / `gensim`; PyTorch, gensim, scikit-learn. Kneser–Ney follows Chen & Goodman (1999). Code structure adapted from the DSA4213 course notebooks. **AI assistance:** I used Claude Code (Anthropic) as a coding assistant for scaffolding, debugging and running experiments; all numbers come from the submitted code, and I verified and wrote the analysis myself.

Appendix

Word	LSTM (random, learned)	Skip-gram (own)	GloVe 6B
king	posse, designer, bishop, schoolmaster	veera, æthelred, leinster, ballala	queen, prince, monarch, kingdom
war	mart, [non-Latin] kalachuris, 1850	outbreak, civil, liberties, airbase	wars, conflict, battle, civil
football	ace, universal, joker, jallon	athletic, f.c., premier, varsity	soccer, basketball, league, baseball
france	outreach, blazers, chivo, cups	italy, germany, sweden, russia	french, paris, belgium, spain
album	crush, roxas, predict, carey	slayer, albums, yankovic, furtado	albums, song, ep, band
good	eaten, 1972, radial, outstandingly	sublime, shocking, exciting, naive	better, really, always, you

Table 5: Cosine nearest neighbours in the three 300-d input-embedding tables (LSTM column: Part I LSTM after training).

Reproduction. `scripts/run_all.sh` regenerates every number, table and figure; all tables in this report are produced by `src/latex_tables.py` from `results/`.

References. Chen & Goodman (1999), An empirical study of smoothing techniques for language modeling, *CSL*. Merity et al. (2017), Pointer sentinel mixture models, *ICLR*. Pennington et al. (2014), GloVe, *EMNLP*. Socher et al. (2013), Recursive deep models for semantic compositionality, *EMNLP*. Zhang et al. (2015), Character-level convolutional networks for text classification, *NeurIPS*.